SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week6 – Wednesday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Hallticket	2303A54038	**Batch**	48
Assignment Number:12.3(Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*Expected Time to complete*
1	**Lab 12: Algorithms with AI Assistance Sorting, Searching, and Algorithm Optimization Using AI Tools** **Lab Objectives** The objectives of this laboratory exercise are to: • Apply AI-assisted programming techniques to implement sorting and searching algorithms. • Analyze and compare algorithm efficiency using time and space complexity. • Understand how AI tools can suggest optimizations and alternative algorithmic approaches. • Strengthen problem-solving skills through real-world, data-driven scenarios. **Learning Outcomes** After completing this lab, students will be able to: • Implement and optimize classic algorithms using AI-assisted coding tools. • Compare multiple algorithms for the same problem and justify their selection. • Measure and analyze runtime performance using experimental data. • Critically review and refine AI-generated algorithmic solutions. **Task 1: Sorting Student Records for Placement Drive** **Scenario** SR University's Training and Placement Cell needs to shortlist candidates	Week3 - Wednesday

efficiently during campus placements. Student records must be sorted by **CGPA in descending order**.

Tasks

1. Use **GitHub Copilot** to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
   - Quick Sort
   - Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the **top 10 students** based on CGPA.

12.3.py

12.3.py > quick_sort

```python
# Write a program to manage student records with Name, Roll Number, and CGPA. Implement Quick Sort and Merge Sort to sort students in desce
from heapq import merge
import time
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa

    def __str__(self):
        return f"{self.name} (Roll No: {self.roll_number}, CGPA: {self.cgpa})"
def quick_sort(students):
    if len(students) <= 1:
        return students
    pivot = students[len(students) // 2].cgpa
    left = [x for x in students if x.cgpa > pivot]
    middle = [x for x in students if x.cgpa == pivot]
    right = [x for x in students if x.cgpa < pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(students):
    if len(students) <= 1:
        return students
    mid = len(students) // 2
    left_half = merge_sort(students[:mid])
    right_half = merge_sort(students[mid:])
    return merge(left_half, right_half)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].cgpa > right[j].cgpa:
            result.append(left[i])
            i += 1
        else:
```

```python
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def main():
    students = [
        Student("Alice", 1, 3.8),
        Student("Bob", 2, 3.5),
        Student("Charlie", 3, 3.9),
        Student("David", 4, 3.6),
        Student("Eve", 5, 3.7),
        Student("Frank", 6, 3.4),
        Student("Grace", 7, 3.9),
        Student("Heidi", 8, 3.2),
        Student("Ivan", 9, 3.8),
        Student("Judy", 10, 3.5)
    ]
    # Measure execution time for Quick Sort
    start_time = time.time()
    sorted_students_quick = quick_sort(students)
    end_time = time.time()
    print(f"Quick Sort Execution Time: {end_time - start_time:.6f} seconds")
    # Measure execution time for Merge Sort
    start_time = time.time()
    sorted_students_merge = merge_sort(students)
    end_time = time.time()
    print(f"Merge Sort Execution Time: {end_time - start_time:.6f} seconds")
    # Display top 10 students with highest CGPA
    print("\nTop 10 Students with Highest CGPA:")
    for student in sorted_students_quick[:10]:
        print(student)
if __name__ == "__main__":
    main()
```

Expected Outcome

- Correctly sorted student records.
- Performance comparison between Quick Sort and Merge Sort.
- Clear output of top-performing students.

```
OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    AZURE

(.venv) PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe c:/Users/ADMIN/Downloads/AI/12.3.py
Merge Sort Execution Time: 0.000040 seconds

Top 10 Students with Highest CGPA:
Charlie (Roll No: 3, CGPA: 3.9)
Grace (Roll No: 7, CGPA: 3.9)
Alice (Roll No: 1, CGPA: 3.8)
Ivan (Roll No: 9, CGPA: 3.8)
Eve (Roll No: 5, CGPA: 3.7)
David (Roll No: 4, CGPA: 3.6)
Bob (Roll No: 2, CGPA: 3.5)
Judy (Roll No: 10, CGPA: 3.5)
Frank (Roll No: 6, CGPA: 3.4)
Heidi (Roll No: 8, CGPA: 3.2)
```

Task 2: Implementing Bubble Sort with AI Comments

- **Task**: Write a Python implementation of **Bubble Sort**.
- **Instructions**:
  - Students implement Bubble Sort normally.
  - Ask AI to generate **inline comments explaining key logic** (like swapping, passes, and termination).
  - Request AI to provide **time complexity analysis**.

```python
'''
# Task-2
# Write a Python program to implement Bubble Sort. Add detailed inline comments explaining each step, including comparisons, swapping of el
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all elements in the list
    for i in range(n):
        # Initialize a flag to check if any swapping occurs
        swapped = False
        # Last i elements are already in place, no need to check them
        for j in range(0, n - i - 1):
            # Compare the adjacent elements
            if arr[j] > arr[j + 1]:
                # If the element found is greater than the next element, swap them
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        # If no swapping occurred, the list is already sorted, so we can break out of the loop
        if not swapped:
            break
    return arr
# Time Complexity Analysis:
# Best Case: O(n) - This occurs when the input list is already sorted. The algorithm makes one pass through the
# list to check if it is sorted, and since no swaps are needed, it terminates early.
# Average Case: O(n^2) - This occurs when the input list is in random order. The algorithm will need to make multiple passes through the li
# Worst Case: O(n^2) - This occurs when the input list is sorted in reverse order. The algorithm will need to make n-1 passes through the l
# Example usage
if __name__ == "__main__":
    arr = [64, 34, 25, 12, 22, 11, 90]
    print("Original array:", arr)
    sorted_arr = bubble_sort(arr)
    print("Sorted array:", sorted_arr)
```

- **Expected Output**:
  - A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

```
(.venv) PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe c:/Users/ADMIN/Downloads/AI/12.3.py
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]
(.venv) PS C:\Users\ADMIN\Downloads\AI>
```

Task 3: Quick Sort and Merge Sort Comparison

- **Task:** Implement Quick Sort and Merge Sort using recursion**.**
- **Instructions:**
  - Provide AI with partially completed functions for recursion.
  - Ask AI to complete the missing logic and add docstrings.
  - Compare both algorithms on random, sorted, and reverse-sorted lists.

```python
# Task-3
# Given partially implemented recursive functions for Quick Sort and Merge Sort in Python, complete the missing logic for both algorithms. Add meaningful docstrings explaining the recursion process. Then write code to test and compare their performance
import random
def quick_sort(arr):
    """
    Quick Sort is a divide-and-conquer algorithm that sorts an array by selecting a 'pivot' element and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then sort

    The recursion process involves:
    1. Choosing a pivot element from the array.
    2. Partitioning the array into two sub-arrays: one with elements less than the pivot and one with elements greater than the pivot.
    3. Recursively applying the same process to the sub-arrays until they are sorted.

    Time Complexity:
    - Best Case: O(n log n) - This occurs when the pivot divides the array into two equal halves consistently.
    - Average Case: O(n log n) - This occurs when the pivots are reasonably well-chosen, leading to balanced partitions.
    - Worst Case: O(n^2) - This occurs when the smallest or largest element is always chosen as the pivot, leading to unbalanced partitions (e.g., when the array is already sorted or reverse-sorted).
    """
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(arr):
    """Merge Sort is a divide-and-conquer algorithm that sorts an array by recursively dividing it into two halves, sorting each half, and then merging the sorted halves back together. The recursion process involves:
    1. Dividing the array into two halves.
    2. Recursively sorting each half.
    3. Merging the sorted halves back together.

    Time Complexity:
    - Best Case: O(n log n) - This occurs when the array is divided into two equal halves at each step.
    - Average Case: O(n log n) - This occurs when the array is divided into two halves of roughly equal size at each step.
    - Worst Case: O(n log n) - This occurs when the array is divided into two halves of very different sizes at each step.
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    merged = []
    i, j = 0, 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
# Testing and comparing performance
if __name__ == "__main__":
    # Generate random, sorted, and reverse-sorted lists
    random_list = [random.randint(1, 100) for _ in range(1000)]
    sorted_list = sorted(random_list)
    reverse_sorted_list = sorted(random_list, reverse=True)
```

```python
# Test Quick Sort
print("Testing Quick Sort...")
print("Random List:", quick_sort(random_list.copy()))
print("Sorted List:", quick_sort(sorted_list.copy()))
print("Reverse Sorted List:", quick_sort(reverse_sorted_list.copy()))

# Test Merge Sort
print("\nTesting Merge Sort...")
print("Random List:", merge_sort(random_list.copy()))
print("Sorted List:", merge_sort(sorted_list.copy()))
print("Reverse Sorted List:", merge_sort(reverse_sorted_list.copy()))
```

- **Expected Output:**
    - Working Quick Sort and Merge Sort implementations.
    - AI-generated explanation of average, best, and worst-case complexities.

```
OUTPUT   ISSUES CONSOLE   TERMINAL   PORTS   AZURE                                                                    Python

(.venv) PS C:\Users\KMPDV\Downloads\AI> & c:/Users/KMPDV/Downloads/AI/.venv/Scripts/python.exe c:/Users/KMPDV/Downloads/AI/12.3.py
92, 92, 92, 93, 93, 93, 93, 93, 93, 93, 93, 93, 94, 94, 94, 94, 94, 94, 94, 94, 95, 95, 95, 95, 95, 95, 95, 95, 96, 96, 96, 96, 96, 96, 96, 96, 97, 97, 97, 97, 97, 97, 97, 97, 98, 98, 98, 98, 98, 98, 98, 98, 99, 99, 99, 99, 99, 99, 99, 99, 100, 100, 100, 100, 100, 100, 100
, 100, 100, 100]

Testing Merge Sort...
Random List: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 5, 5, 5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 7, 7, 7, 7, 7, 7, 7, 7, 8, 8, 8, 8, 8, 8, 8, 8, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9,
9, 9, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 11, 11, 11, 11, 11, 11, 11, 11, 12, 12, 12, 12, 12, 12, 12, 12, 13, 13, 13, 13, 13, 13, 13, 13, 14, 14, 14, 14, 14, 14, 14, 14, 15, 15, 15, 15, 15, 15, 15, 15, 16, 16, 16, 16, 16, 16, 16, 16, 17, 17, 17, 17
, 17, 17, 17, 17, 17, 17, 17, 17, 18, 18, 18, 18, 18, 18, 18, 18, 19, 19, 19, 19, 19, 19, 19, 19, 20, 20, 20, 20, 20, 20, 20, 20, 21, 21, 21, 21, 21, 21, 21, 21, 21, 21, 21, 21, 22, 22, 22, 22, 22, 22, 22, 22, 23, 23, 23, 23, 23, 23, 23, 23,
24, 24, 24, 24, 24, 24, 24, 24, 25, 25, 25, 25, 25, 25, 25, 25, 26, 26, 26, 26, 26, 26, 26, 26, 27, 27, 27, 27, 27, 27, 27, 27, 28, 28, 28, 28, 28, 28, 28, 28, 29, 29, 29, 29, 29, 29, 29, 29, 30, 30, 30, 30, 30, 30, 30, 30,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 33, 33, 33, 33, 34, 34, 34, 34, 34, 34, 34, 34, 35, 35, 35, 35, 35, 35, 35, 35, 36, 36, 36, 36, 36, 36, 36, 36, 37, 37, 37, 37, 3
7, 37, 37, 37, 37, 37, 37, 37, 37, 38, 38, 38, 38, 38, 38, 38, 38, 39, 39, 39, 39, 39, 39, 39, 39, 40, 40, 40, 40, 40, 40, 40, 40, 41, 41, 41, 41, 41, 41, 41, 41, 42, 42, 42, 42, 42, 42, 42, 42, 43, 43, 43, 43, 43, 43, 43, 43, 44, 44, 44, 44, 44, 44, 44, 44,
, 44, 44, 44, 44, 45, 45, 45, 45, 45, 45, 45, 45, 46, 46, 46, 46, 46, 46, 46, 46, 47, 47, 47, 47, 47, 47, 47, 47, 48, 48, 48, 48, 48, 48, 48, 48, 49, 49, 49, 49, 49, 49, 49, 49, 50, 50, 50, 50, 50, 50, 50, 50, 51, 51, 51, 51, 51, 51, 51, 52, 52, 52, 52, 52, 52, 52, 53, 53, 53,
53, 53, 53, 53, 53, 53, 53, 53, 54, 54, 54, 54, 54, 54, 54, 54, 55, 55, 55, 55, 55, 55, 55, 55, 56, 56, 56, 56, 56, 56, 56, 56, 57, 57, 57, 57, 57, 57, 57, 57, 58, 58, 58, 58, 58, 58, 58, 58, 59, 59, 59, 59, 59, 59, 59, 59, 60, 60, 60,
60, 60, 60, 60, 60, 61, 61, 61, 61, 61, 61, 61, 61, 62, 62, 62, 62, 62, 62, 62, 62, 63, 63, 63, 63, 63, 63, 63, 63, 64, 64, 64, 64, 64, 64, 64, 64, 65, 65, 65, 65, 65, 65, 65, 65, 66, 66, 66, 66, 66, 66, 66, 66, 67, 67, 67, 67, 67, 67, 67, 67, 68, 68, 68, 68, 68, 6
8, 68, 68, 69, 69, 69, 69, 69, 69, 69, 69, 70, 70, 70, 70, 70, 70, 70, 70, 71, 71, 71, 71, 71, 71, 71, 71, 72, 72, 72, 72, 72, 72, 72, 72, 73, 73, 73, 73, 73, 73, 73, 73, 74, 74, 74, 74, 74, 74, 74, 74, 75, 75, 75, 75, 75, 75, 75, 75,
, 75, 75, 75, 76, 76, 76, 76, 76, 76, 76, 76, 77, 77, 77, 77, 77, 77, 77, 77, 78, 78, 78, 78, 78, 78, 78, 78, 79, 79, 79, 79, 79, 79, 79, 79, 80, 80, 80, 80, 80, 80, 80, 80, 81, 81, 81, 81, 81, 81, 81, 81, 82, 82, 82, 82, 82, 82, 82, 82, 83, 83, 84, 86,
84, 84, 84, 84, 84, 84, 84, 84, 85, 85, 85, 85, 85, 85, 85, 85, 86, 86, 86, 86, 86, 86, 86, 86, 87, 87, 87, 87, 87, 87, 87, 87, 88, 88, 88, 88, 88, 88, 88, 88, 89, 89, 89, 89, 89, 89, 89, 89, 90, 90, 90, 90, 90, 90, 90, 90, 91, 91, 91, 91, 91, 91, 91, 91, 92, 92, 92, 92, 92, 92, 92, 92,
92, 93, 93, 93, 93, 93, 93, 93, 93, 93, 94, 94, 94, 94, 94, 94, 94, 94, 95, 95, 95, 95, 95, 95, 95, 95, 96, 96, 96, 96, 96, 96, 96, 96, 97, 97, 97, 97, 97, 97, 97, 97, 98, 98, 98, 98, 98, 98, 98, 98, 99, 99, 99, 99, 99, 99, 99, 99, 100, 100, 100, 100, 100, 100, 100, 100
, 100, 100]
Sorted List: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 5, 5, 5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 7, 7, 7, 7, 7, 7, 7, 7, 8, 8, 8, 8, 8, 8, 8, 8, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9,
9, 9, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 11, 11, 11, 11, 11, 11, 11, 11, 12, 12, 12, 12, 12, 12, 12, 12, 13, 13, 13, 13, 13, 13, 13, 13, 14, 14, 14, 14, 14, 14, 14, 14, 15, 15, 15, 15, 15, 15, 15, 15, 16, 16, 16, 16, 16, 16, 16, 16, 17, 17, 17, 17
, 17, 17, 17, 17, 17, 17, 17, 17, 18, 18, 18, 18, 18, 18, 18, 18, 19, 19, 19, 19, 19, 19, 19, 19, 20, 20, 20, 20, 20, 20, 20, 20, 21, 21, 21, 21, 21, 21, 21, 21, 21, 21, 21, 21, 22, 22, 22, 22, 22, 22, 22, 22, 23, 23, 23, 23, 23, 23, 23, 23,
24, 24, 24, 24, 24, 24, 24, 24, 25, 25, 25, 25, 25, 25, 25, 25, 26, 26, 26, 26, 26, 26, 26, 26, 27, 27, 27, 27, 27, 27, 27, 27, 28, 28, 28, 28, 28, 28, 28, 28, 29, 29, 29, 29, 29, 29, 29, 29, 30, 30, 30, 30, 30, 30, 30, 30,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 33, 33, 33, 33, 34, 34, 34, 34, 34, 34, 34, 34, 35, 35, 35, 35, 35, 35, 35, 35, 36, 36, 36, 36, 36, 36, 36, 36, 37, 37, 37, 37, 3
7, 37, 37, 37, 37, 37, 37, 37, 37, 38, 38, 38, 38, 38, 38, 38, 38, 39, 39, 39, 39, 39, 39, 39, 39, 40, 40, 40, 40, 40, 40, 40, 40, 41, 41, 41, 41, 41, 41, 41, 41, 42, 42, 42, 42, 42, 42, 42, 42, 43, 43, 43, 43, 43, 43, 43, 43, 44, 44, 44, 44, 44, 44, 44, 44,
, 44, 44, 44, 44, 45, 45, 45, 45, 45, 45, 45, 45, 46, 46, 46, 46, 46, 46, 46, 46, 47, 47, 47, 47, 47, 47, 47, 47, 48, 48, 48, 48, 48, 48, 48, 48, 49, 49, 49, 49, 49, 49, 49, 49, 50, 50, 50, 50, 50, 50, 50, 50, 51, 51, 51, 51, 51, 51, 51, 52, 52, 52, 52, 52, 52, 52, 53, 53, 53,
53, 53, 53, 53, 53, 53, 53, 53, 54, 54, 54, 54, 54, 54, 54, 54, 55, 55, 55, 55, 55, 55, 55, 55, 56, 56, 56, 56, 56, 56, 56, 56, 57, 57, 57, 57, 57, 57, 57, 57, 58, 58, 58, 58, 58, 58, 58, 58, 59, 59, 59, 59, 59, 59, 59, 59, 60, 60, 60,
60, 60, 60, 60, 60, 61, 61, 61, 61, 61, 61, 61, 61, 62, 62, 62, 62, 62, 62, 62, 62, 63, 63, 63, 63, 63, 63, 63, 63, 64, 64, 64, 64, 64, 64, 64, 64, 65, 65, 65, 65, 65, 65, 65, 65, 66, 66, 66, 66, 66, 66, 66, 66, 67, 67, 67, 67, 67, 67, 67, 67, 68, 68, 68, 68, 68, 6
8, 68, 68, 69, 69, 69, 69, 69, 69, 69, 69, 70, 70, 70, 70, 70, 70, 70, 70, 71, 71, 71, 71, 71, 71, 71, 71, 72, 72, 72, 72, 72, 72, 72, 72, 73, 73, 73, 73, 73, 73, 73, 73, 74, 74, 74, 74, 74, 74, 74, 74, 75, 75, 75, 75, 75, 75, 75, 75,
, 75, 75, 75, 76, 76, 76, 76, 76, 76, 76, 76, 77, 77, 77, 77, 77, 77, 77, 77, 78, 78, 78, 78, 78, 78, 78, 78, 79, 79, 79, 79, 79, 79, 79, 79, 80, 80, 80, 80, 80, 80, 80, 80, 81, 81, 81, 81, 81, 81, 81, 81, 82, 82, 82, 82, 82, 82, 82, 82, 83, 83, 84, 86,
84, 84, 84, 84, 84, 84, 84, 84, 85, 85, 85, 85, 85, 85, 85, 85, 86, 86, 86, 86, 86, 86, 86, 86, 87, 87, 87, 87, 87, 87, 87, 87, 88, 88, 88, 88, 88, 88, 88, 88, 89, 89, 89, 89, 89, 89, 89, 89, 90, 90, 90, 90, 90, 90, 90, 90, 91, 91, 91, 91, 91, 91, 91, 91, 92, 92, 92, 92, 92, 92, 92, 92,
92, 93, 93, 93, 93, 93, 93, 93, 93, 93, 94, 94, 94, 94, 94, 94, 94, 94, 95, 95, 95, 95, 95, 95, 95, 95, 96, 96, 96, 96, 96, 96, 96, 96, 97, 97, 97, 97, 97, 97, 97, 97, 98, 98, 98, 98, 98, 98, 98, 98, 99, 99, 99, 99, 99, 99, 99, 99, 100, 100, 100, 100, 100, 100, 100, 100
, 100, 100]
Reverse Sorted List: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 5, 5, 5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 7, 7, 7, 7, 7, 7, 7, 7, 8, 8, 8, 8, 8, 8, 8, 8, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9,
9, 9, 9, 9, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 11, 11, 11, 11, 11, 11, 11, 11, 12, 12, 12, 12, 12, 12, 12, 12, 13, 13, 13, 13, 13, 13, 13, 13, 14, 14, 14, 14, 14, 14, 14, 14, 15, 15, 15, 15, 15, 15, 15, 15, 16, 16, 16, 16, 16, 16, 16, 16, 17, 17, 17, 17
, 17, 17, 17, 17, 17, 17, 17, 17, 18, 18, 18, 18, 18, 18, 18, 18, 19, 19, 19, 19, 19, 19, 19, 19, 20, 20, 20, 20, 20, 20, 20, 20, 21, 21, 21, 21, 21, 21, 21, 21, 21, 21, 21, 21, 22, 22, 22, 22, 22, 22, 22, 22, 23, 23, 23, 23, 23, 23, 23, 23,
24, 24, 24, 24, 24, 24, 24, 24, 25, 25, 25, 25, 25, 25, 25, 25, 26, 26, 26, 26, 26, 26, 26, 26, 27, 27, 27, 27, 27, 27, 27, 27, 28, 28, 28, 28, 28, 28, 28, 28, 29, 29, 29, 29, 29, 29, 29, 29, 30, 30, 30, 30, 30, 30, 30, 30,
31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 32, 32, 33, 33, 33, 33, 33, 33, 33, 33, 34, 34, 34, 34, 34, 34, 34, 34, 35, 35, 35, 35, 35, 35, 35, 35, 36, 36, 36, 36, 36, 36, 36, 36, 37, 37, 37, 37, 3
7, 37, 37, 37, 37, 37, 37, 37, 37, 38, 38, 38, 38, 38, 38, 38, 38, 39, 39, 39, 39, 39, 39, 39, 39, 40, 40, 40, 40, 40, 40, 40, 40, 41, 41, 41, 41, 41, 41, 41, 41, 42, 42, 42, 42, 42, 42, 42, 42, 43, 43, 43, 43, 43, 43, 43, 43, 44, 44, 44, 44, 44, 44, 44, 44,
, 44, 44, 44, 44, 45, 45, 45, 45, 45, 45, 45, 45, 46, 46, 46, 46, 46, 46, 46, 46, 47, 47, 47, 47, 47, 47, 47, 47, 48, 48, 48, 48, 48, 48, 48, 48, 49, 49, 49, 49, 49, 49, 49, 49, 50, 50, 50, 50, 50, 50, 50, 50, 51, 51, 51, 51, 51, 51, 51, 52, 52, 52, 52, 52, 52, 52, 53, 53, 53,
53, 53, 53, 53, 53, 53, 53, 53, 54, 54, 54, 54, 54, 54, 54, 54, 55, 55, 55, 55, 55, 55, 55, 55, 56, 56, 56, 56, 56, 56, 56, 56, 57, 57, 57, 57, 57, 57, 57, 57, 58, 58, 58, 58, 58, 58, 58, 58, 59, 59, 59, 59, 59, 59, 59, 59, 60, 60, 60,
60, 60, 60, 60, 60, 61, 61, 61, 61, 61, 61, 61, 61, 62, 62, 62, 62, 62, 62, 62, 62, 63, 63, 63, 63, 63, 63, 63, 63, 64, 64, 64, 64, 64, 64, 64, 64, 65, 65, 65, 65, 65, 65, 65, 65, 66, 66, 66, 66, 66, 66, 66, 66, 67, 67, 67, 67, 67, 67, 67, 67, 68, 68, 68, 68, 68, 6
8, 68, 68, 69, 69, 69, 69, 69, 69, 69, 69, 70, 70, 70, 70, 70, 70, 70, 70, 71, 71, 71, 71, 71, 71, 71, 71, 72, 72, 72, 72, 72, 72, 72, 72, 73, 73, 73, 73, 73, 73, 73, 73, 74, 74, 74, 74, 74, 74, 74, 74, 75, 75, 75, 75, 75, 75, 75, 75,
, 75, 75, 75, 76, 76, 76, 76, 76, 76, 76, 76, 77, 77, 77, 77, 77, 77, 77, 77, 78, 78, 78, 78, 78, 78, 78, 78, 79, 79, 79, 79, 79, 79, 79, 79, 80, 80, 80, 80, 80, 80, 80, 80, 81, 81, 81, 81, 81, 81, 81, 81, 82, 82, 82, 82, 82, 82, 82, 82, 83, 83, 84, 86,
84, 84, 84, 84, 84, 84, 84, 84, 85, 85, 85, 85, 85, 85, 85, 85, 86, 86, 86, 86, 86, 86, 86, 86, 87, 87, 87, 87, 87, 87, 87, 87, 88, 88, 88, 88, 88, 88, 88, 88, 89, 89, 89, 89, 89, 89, 89, 89, 90, 90, 90, 90, 90, 90, 90, 90, 91, 91, 91, 91, 91, 91, 91, 91, 92, 92, 92, 92, 92, 92, 92, 92,
92, 93, 93, 93, 93, 93, 93, 93, 93, 93, 94, 94, 94, 94, 94, 94, 94, 94, 95, 95, 95, 95, 95, 95, 95, 95, 96, 96, 96, 96, 96, 96, 96, 96, 97, 97, 97, 97, 97, 97, 97, 97, 98, 98, 98, 98, 98, 98, 98, 98, 99, 99, 99, 99, 99, 99, 99, 99, 100, 100, 100, 100, 100, 100, 100, 100
, 100, 100, 100]
(.venv) PS C:\Users\KMPDV\Downloads\AI>
```

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.

```python
# Task-4
# Design a Python-based inventory management system for a retail store with product attributes: product ID, name, price, and stock quantity. Suggest and implement e
class Product:
    def __init__(self, product_id, name, price, stock_quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock_quantity = stock_quantity

    def __str__(self):
        return f"{self.name} (ID: {self.product_id}, Price: {self.price}, Stock: {self.stock_quantity})"
class Inventory:
    def __init__(self):
        self.products = []

    def add_product(self, product):
        self.products.append(product)

    def search_by_id(self, product_id):
        """Search for a product by its ID using binary search. This is efficient for large datasets that are sorted by product ID."""
        self.products.sort(key=lambda x: x.product_id)  # Ensure products are sorted by ID
        left, right = 0, len(self.products) - 1
        while left <= right:
            mid = (left + right) // 2
            if self.products[mid].product_id == product_id:
                return self.products[mid]
            elif self.products[mid].product_id < product_id:
                left = mid + 1
            else:
                right = mid - 1
        return None

    def search_by_name(self, name):
        """Search for a product by its name using linear search. This is suitable for small datasets or when the dataset is frequently updated."""
        for product in self.products:
            if product.name.lower() == name.lower():
                return product
```

```python
    def sort_by_price(self):
        """Sort products by price using Timsort (Python's built-in sorting algorithm), which is efficient for large datasets and partially sorted data."""
        self.products.sort(key=lambda x: x.price)

    def sort_by_stock_quantity(self):
        """Sort products by stock quantity using Timsort, which is efficient for large datasets and partially sorted data."""
        self.products.sort(key=lambda x: x.stock_quantity)
# Comparison Table
# | Operation            | Recommended Algorithm | Justification                |
# |----------------------|-----------------------|------------------------------|
# | Search by ID         | Binary Search         | Efficient for sorted data    |
# | Search by Name       | Linear Search         | Suitable for small datasets or frequent updates |
# | Sort by Price        | Timsort (Python's built-in) | Efficient for large datasets and partially sorted data |
# | Sort by Stock Quantity | Timsort (Python's built-in) | Efficient for large datasets and partially sorted data |
# Example usage
if __name__ == "__main__":
    inventory = Inventory()
    inventory.add_product(Product(1, "Laptop", 999.99, 10))
    inventory.add_product(Product(2, "Smartphone", 499.99, 20))
    inventory.add_product(Product(3, "Headphones", 199.99, 15))

    # Search by ID
    print("Search by ID (2):", inventory.search_by_id(2))

    # Search by Name
    print("Search by Name (Headphones):", inventory.search_by_name("Headphones"))

    # Sort by Price
    inventory.sort_by_price()
    print("\nProducts sorted by price:")
    for product in inventory.products:
        print(product)

    # Sort by Stock Quantity
    inventory.sort_by_stock_quantity()
    print("\nProducts sorted by stock quantity:")
    for product in inventory.products:
        print(product)
```

Expected Output:

- A table mapping operation → recommended algorithm → justification.
- Working Python functions for searching and sorting the inventory.

```
(.venv) PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe c:/Users/ADMIN/Downloads/AI/12.3.py
Search by ID (2): Smartphone (ID: 2, Price: 499.99, Stock: 20)
Search by Name (Headphones): Headphones (ID: 3, Price: 199.99, Stock: 15)

Products sorted by price:
Headphones (ID: 3, Price: 199.99, Stock: 15)
Smartphone (ID: 2, Price: 499.99, Stock: 20)
Laptop (ID: 1, Price: 999.99, Stock: 10)

Products sorted by stock quantity:
Laptop (ID: 1, Price: 999.99, Stock: 10)
Headphones (ID: 3, Price: 199.99, Stock: 15)
Smartphone (ID: 2, Price: 499.99, Stock: 20)
(.venv) PS C:\Users\ADMIN\Downloads\AI>
```

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered **FinTech Lab** at SR University is building a tool for analyzing **stock price movements**. The requirement is to quickly **sort stocks by daily gain/loss** and search for specific stock symbols efficiently.

- Use **GitHub Copilot** to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).

- Implement sorting algorithms to rank stocks by **percentage change**.
- Implement a **search function** that retrieves stock data instantly when a stock symbol is entered.
- Optimize sorting with **Heap Sort** and searching with **Hash Maps**.
- Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

```python
# Task 5
# Develop a Python-based stock analysis tool that generates or simulates stock data (stock symbol, opening price, closing price). Compute percentage change and sort stocks using Heap Sort based on gain/loss. Implement efficie
from heapq import heapify
import random
import time
class Stock:
    def __init__(self, symbol, opening_price, closing_price):
        self.symbol = symbol
        self.opening_price = opening_price
        self.closing_price = closing_price

    def percentage_change(self):
        return ((self.closing_price - self.opening_price) / self.opening_price) * 100

    def __str__(self):
        return f"{self.symbol} (Open: {self.opening_price}, Close: {self.closing_price}, Change: {self.percentage_change():.2f}%)"
def heap_sort(stocks):
    """Heap Sort is a comparison-based sorting algorithm that uses a binary heap data structure to sort elements. It works by building a max heap from the input data and then repeatedly extracting the maximum element from th

    Time Complexity:
    - Best Case: O(n log n) - This occurs when the input data is in random order.
    - Average Case: O(n log n) - This occurs when the input data is in random order.
    - Worst Case: O(n log n) - This occurs when the input data is in random order, as Heap Sort does not take advantage of any existing order in the data.
    """
    n = len(stocks)
    # Build a maxheap
    for i in range(n // 2 - 1, -1, -1):
        heapify(stocks, n, i)
    # One by one extract elements from heap
    for i in range(n - 1, 0, -1):
        stocks[i], stocks[0] = stocks[0], stocks[i]  # swap
        heapify(stocks, i, 0)
def hash_map_search(stocks, symbol):
    """Search for a stock by its symbol using a hash map (dictionary in Python). This allows for O(1) average time complexity for lookups, making it efficient for large datasets."""
    stock_dict = {stock.symbol: stock for stock in stocks}
    return stock_dict.get(symbol, None)
def linear_search(stocks, symbol):
    """Search for a stock by its symbol using linear search. This has O(n) time complexity, which is less efficient than hash map search, especially for large datasets."""
    for stock in stocks:
        if stock.symbol == symbol:
            return stock
    return None
# Generate simulated stock data
def generate_stock_data(num_stocks):
    stocks = []
    for i in range(num_stocks):
        symbol = f"STK{i}"
        opening_price = random.uniform(100, 500)
```

```python
def generate_stock_data(num_stocks):
        closing_price = opening_price + random.uniform(-50, 50)
        stocks.append(Stock(symbol, opening_price, closing_price))
    return stocks
# Main function to test and compare performance
if __name__ == "__main__":
    num_stocks = 1000
    stocks = generate_stock_data(num_stocks)
    # Measure execution time for Heap Sort
    start_time = time.time()
    heap_sort(stocks)
    end_time = time.time()
    print(f"Heap Sort Time: {end_time - start_time:.6f} seconds")

    # Measure execution time for sorted()
    start_time = time.time()
    stocks_sorted = sorted(stocks, key=lambda x: x.percentage_change(), reverse=True)
    end_time = time.time()
    print(f"sorted() Time: {end_time - start_time:.6f} seconds")

    # Measure execution time for Hash Map Search
    start_time = time.time()
    stock = hash_map_search(stocks, "STK500")
    end_time = time.time()
    print(f"Hash Map Search Time: {end_time - start_time:.6f} seconds")

    # Measure execution time for Linear Search
    start_time = time.time()
    stock = linear_search(stocks, "STK500")
    end_time = time.time()
    print(f"Linear Search Time: {end_time - start_time:.6f} seconds")
    # Display top gainers and losers
    print("\nTop Gainers:")
    for stock in stocks_sorted[:10]:
        print(stock)
    print("\nTop Losers:")
    for stock in stocks_sorted[-10:]:
        print(stock)
```

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

```
(.venv) PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe c:/Users/ADMIN/Downloads/AI/12.3.py
Heap Sort Execution Time: 0.000035 seconds
Built-in sorted() Execution Time: 0.000047 seconds

Top Gainers:
AAPL: 180.71%
AMZN: 95.60%
GOOGL: 41.79%

Top Losers:
GOOGL: 41.79%
TSLA: -42.28%
MSFT: -45.63%

Searching for AAPL using Hash Map:
AAPL: 180.71%

Searching for AAPL using Linear Search:
AAPL: 180.71%
(.venv) PS C:\Users\ADMIN\Downloads\AI>
```